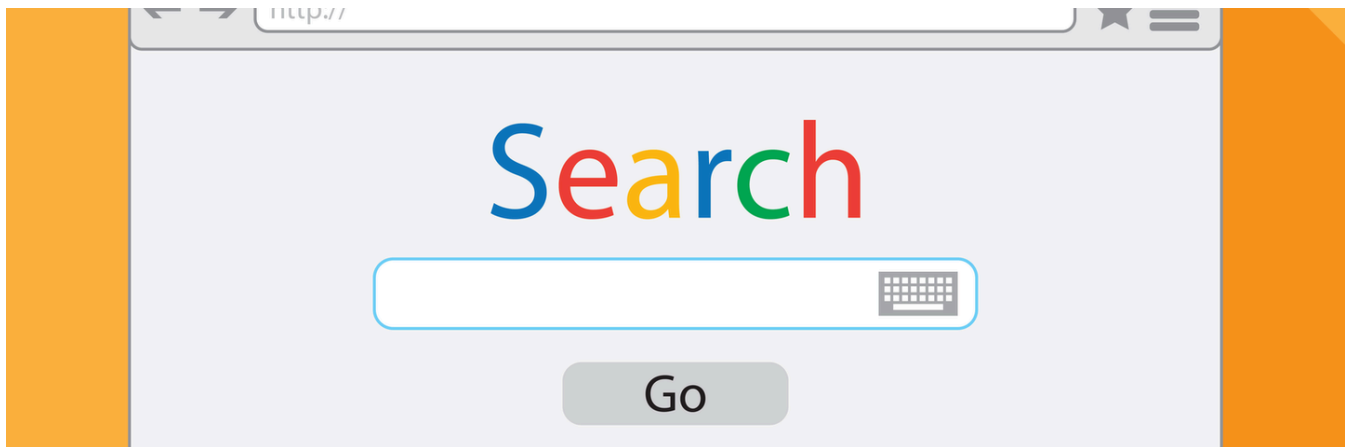




ANN 召回算法之 IVFPQ



TSLA 多头

📖 收录于 · ANN 索引

259 人赞同了该文章

ANN⁺ (Approximate Nearest Neighbor), 顾名思义, 找出大致相近的结果。个性化推荐系统中, 常见的流程为 召回->粗排->精排->重排四阶段。当前的召回阶段通常为向量化召回, 即一切皆用 embedding 来表示。

举例广告投放系统, 我们所要关注的是当前用户适合哪些广告(比如我喜欢在淘宝上买书, 那么淘宝给我推送的广告更多的是书籍相关)。举例常见的双塔模型(用户塔*广告塔), 离线侧通过广告塔计算出全量广告的 embedding; 在线侧用户请求打过来之后, 抽取用户特征(比如用户喜欢买书)通过用户塔计算出当前用户的 embedding。通过计算两个 embedding 的相似度(即两个 embedding 的距离, 例如余弦距离)来标识当前用户和当前广告的 match 程度。获取与当前用户 Top K 条最符合的广告之后, 召回流程结束, 随后便是后续的打分排序环节。

那么我们如何找到与当前用户最相近的 Top K 条广告呢? 本质很简单, 假设我现在有十条广告, 我有他们的 embedding, 那么对于我这个用户的 embedding, 我只需要跟这十条广告分别计算向量距离(比如向量内积), 此时就计算出 10 个距离, 按照距离从近到远排序, 便能够知道哪几条广告跟我最 match。然而, 现实场景中, 但凡一个 APP 的 DAU 足够大, 肯定有上千万甚至上亿的广告在这个平台投放。我跟 10 个广告进行 match 没啥问题, 那我一个请求过来, 我跟几千万条广告去 match, 岂不是 match 到明年。

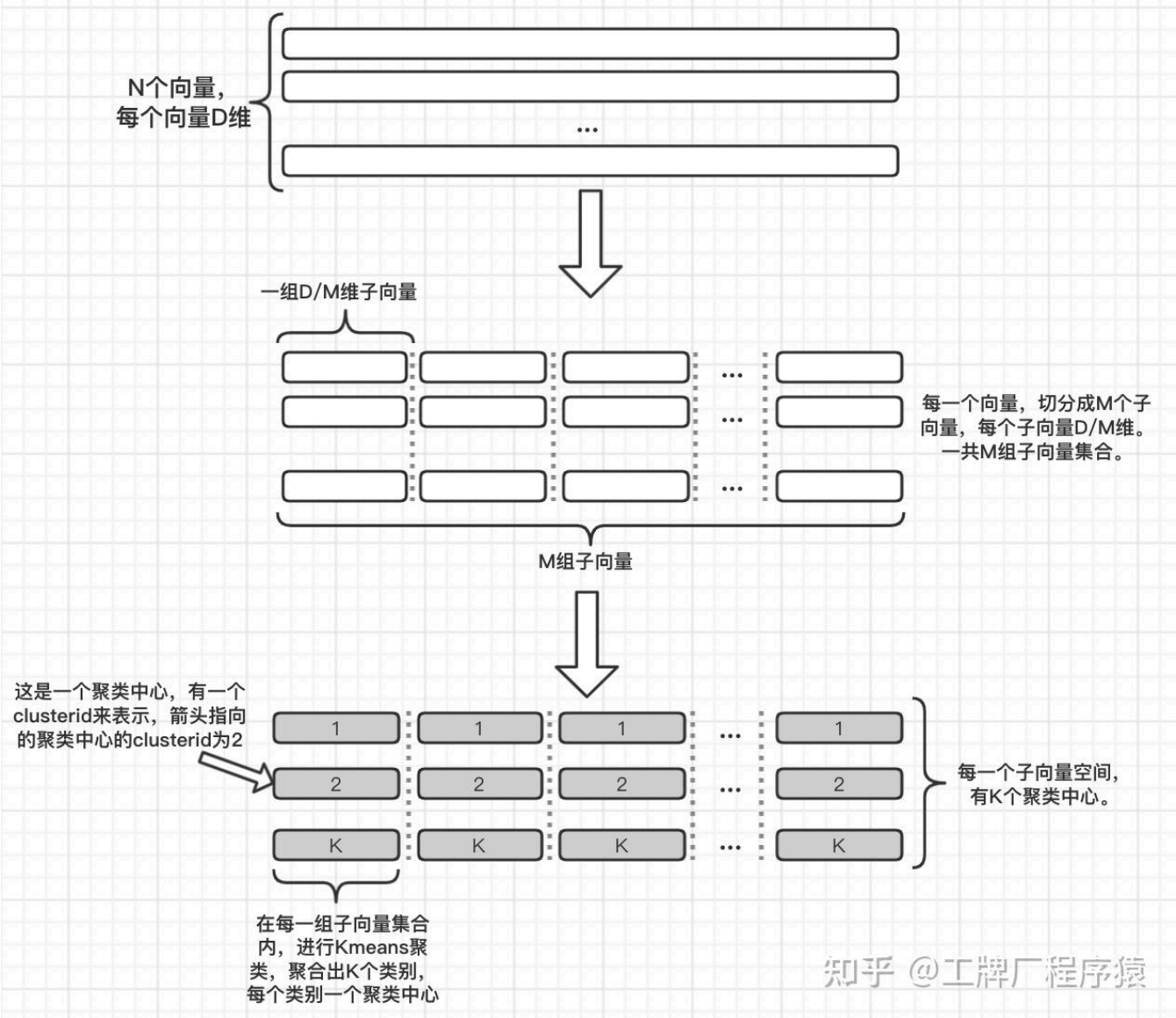
我们刚刚所说的一个用户跟 10 条广告 match 的过程, 就是 **brute-force⁺**, 俗称暴力计算。我跟所有的广告一个个 match 过来, 精确度绝对是最高的, 因为我 100% 精准。也是最低效的, 因为我干了很多无用功。除此之外, 还有个非常严重的问题, 那就是存储。假设我一条广告, 用一个 1024 维的 float 向量来表示, 那么我一条广告占用多少空间? 我占用 **4Byte * 1024** 个字节(float 四个字节哈)。那我有一千万条广告呢? **1000W * 4Byte * 1024 = 38G**。

暴力计算通常用于精准度要求极高的场景, 比如公安局人脸搜索啥的。但是在个性化推荐场景下, 我们要的是个性化, 而不是绝对精准, 也就是说要快速且大致精准。

总结一下一共两个问题, 一是如何快速找到 Top 相近的(毫秒级), 二是如何高效的存储向量。解决方案就是缩小查找范围, 以及向量压缩。这里笔者来谈谈业界常见的一种 ANN 算法: IVFPQ。本文会围绕一篇 paper 来讲:

https://lear.inrialpes.fr/pubs/2011/JDS11/jegou_searching_with_quantization.pdf
lear.inrialpes.fr/pubs/2011/JDS11/jegou_searching_with_qua

我们先拿一个向量库作为例子。



如图 Figure1 所示。我们有一个向量库(例如广告库)，里面有 N 个向量，每个向量 D 维。第一步我们先进行切分，将 D 维向量切分成 M 组子向量，每个子向量 $\frac{D}{M}$ 维。第二步，我们分别在每一组子向量集合内，做 Kmeans 聚类，在每个子向量空间中，产生 K 个聚类中心。每个聚类中心就是一个 $\frac{D}{M}$ 维子向量，他们由一个 id 来表示，叫做 clusterid。一个子空间中所有的 clusterid，构造了一个属于当前子空间的 codebook。那么对于当前向量库，我们就有 M 个 codebook。那么这 M 个 codebook 所能表示的样本量级就是 K^M ，也就是 M 个 codebook 的笛卡尔积，乘积量化的名字"乘积"就是这么来的。这种量化方式可以表示非常大的空间。那么 K 值和 M 值如何选择呢？Paper 作者做过相关测试。

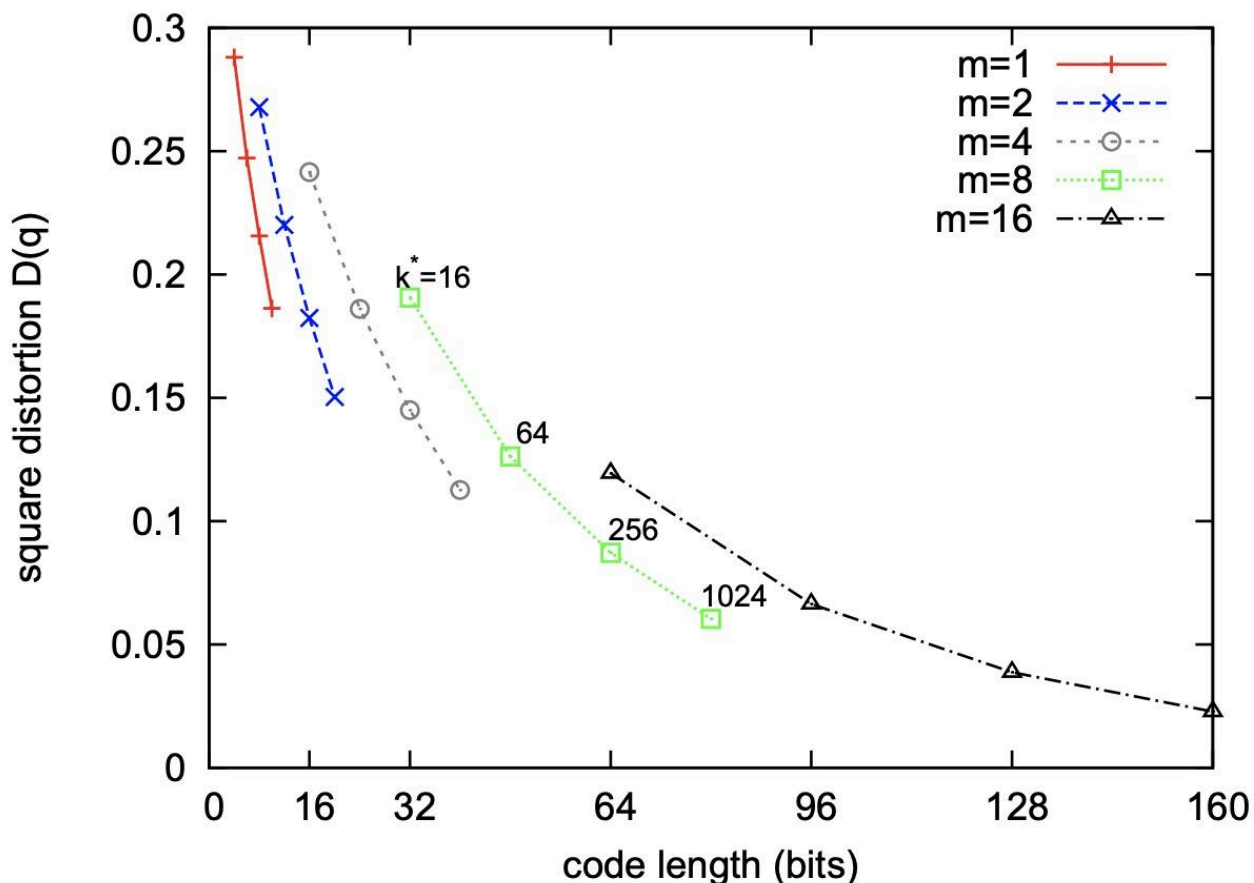


Fig. 1. SIFT: quantization error associated with the parameters m and k^* .

Figure2

如图 Figure2, Paper 作者分别针对几种不同的 K 值和 M 值, 做了测试, 得出这张图表。横轴表示的是一个向量压缩之后占用的存储空间, 纵轴表示的是均方误差(MSE)。比如绿色曲线(M=8), 在 K=256 的时候, 需要 8bit 来表示子向量量化之后的结果(所属的聚类的 clusterid), 那么一个样本量化后的占用空间就是 $8bit * 8 = 64bit$, 此时测试出来的 MSE=0.1。目前业界更多的采用 M=8, K=256 的做法, 权衡占用空间以及误差。

我们的量化函数的目标, 就是为每一个样本, 寻找到距离它最近的那个聚类中心, 即量化函数 $q(x) = \operatorname{argmin}(\operatorname{dist}(x, c))$ 。之后我们所讲的为样本寻找到它所属的聚类中心的过程, 就用 $q(x)$ 来表示。

对于子向量空间中的 N 个子向量样本, 在完成 Kmeans 聚类之后, 我们用这个聚类中心的 clusterid 来代表这个子向量。如图 Figure3 所示。

当前子向量空间中，每一个子向量用其所属的聚类中心的clusterid来表示。比如这个子向量样本，属于聚类中心clusterid=3，那么就用数字3代替之。

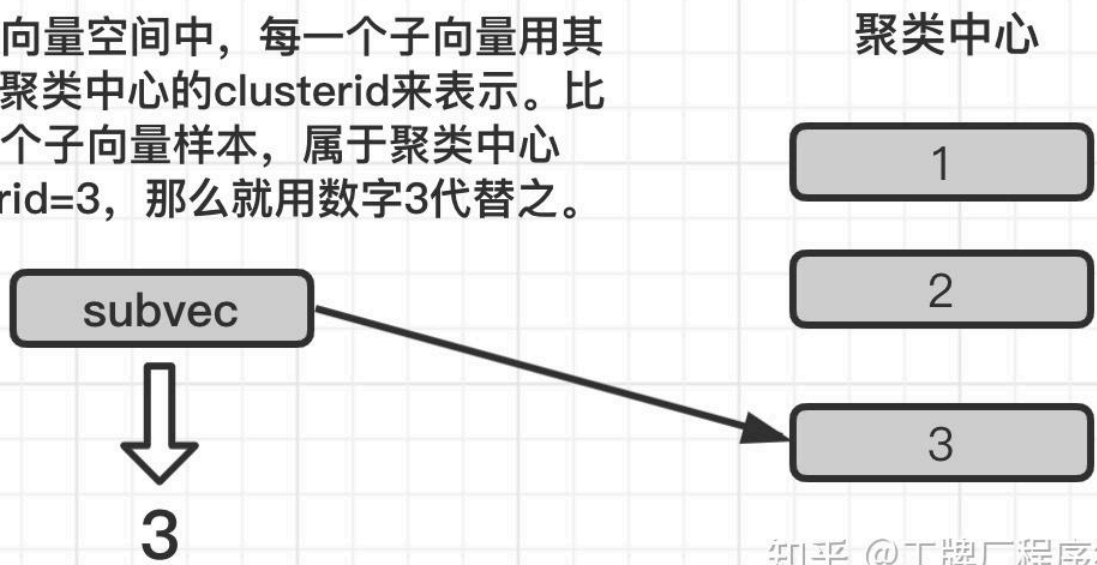


Figure3

那么这样一来，这个向量库，就变成了如图 Figure4 所示的样子。

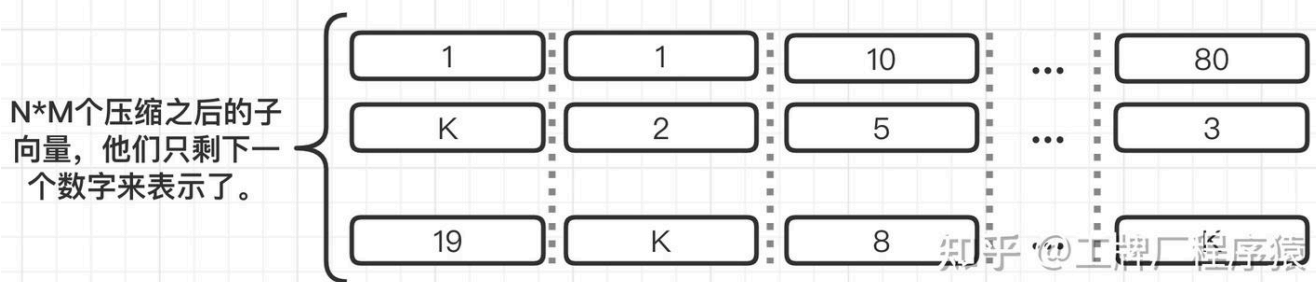


Figure4

原本我们的向量库的大小为 $N * D * 32bit$ ，压缩后，clusterid 按照 8bit 来算的话，那就是 $N * M * 8bit$ ，相比压缩前少了很多。

那么当我们进行向量库检索的时候，该怎么做呢？这里有两种计算方式，分别是 SDC 以及 ADC。Figure5 摘自 Paper。

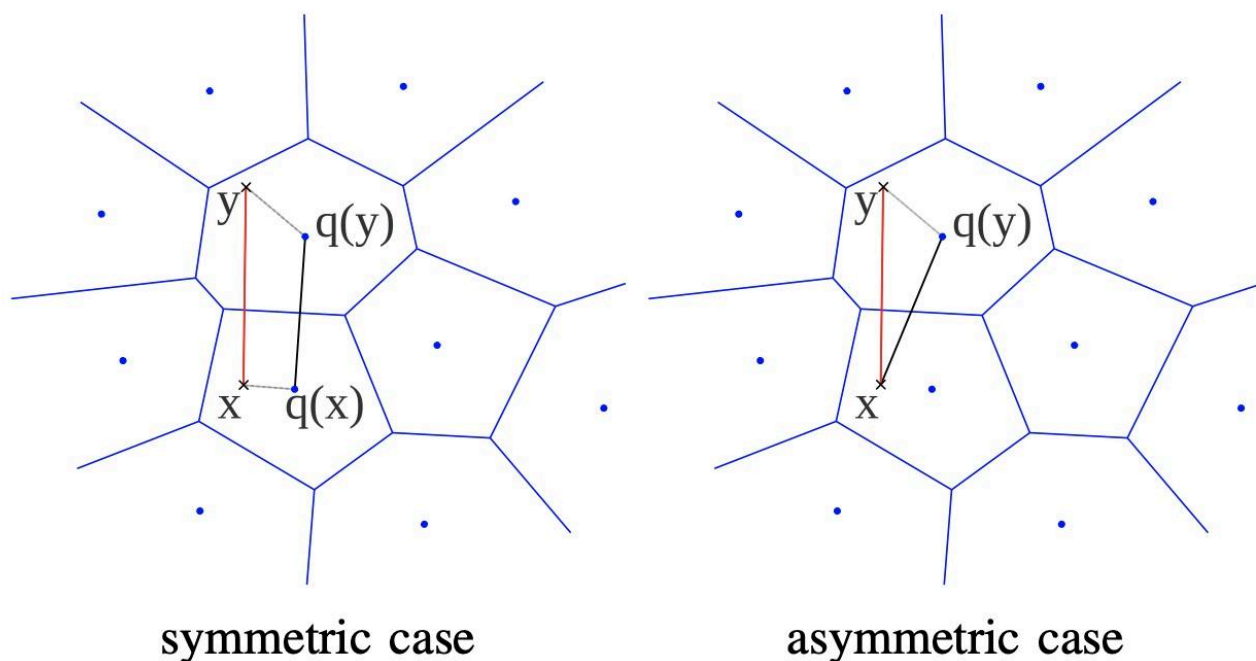


Fig. 2. Illustration of the symmetric and asymmetric distance computation. The distance $d(x, y)$ is estimated with either the distance $d(q(x), q(y))$ (left) or the distance $d(x, q(y))$ (right). The mean squared error on the distance is on average bounded by the quantization error.

知乎 @工牌厂程序猿

Figure5

我们分别来说SDC以及ADC。先说明，接下来笔者所描述的流程，是在一个子向量空间中的流程。

SDC

S=symmetric，对称的。如图Figure5的symmetric case。图中x就是query检索向量，y就是向量库里面的向量(注意，y已经是量化过了的，就是上文中说的那个用数字id替代向量)。那么如何计算x与y的距离呢？首先，计算 $q(x)$ ，拿到x对应的聚类中心；同样的，计算 $q(y)$ ，拿到y对应的聚类中心。 $q(x)$ 和 $q(y)$ 就是两个完整的子向量，我们计算这两个向量的距离，便是当前子空间下的距离。为什么名字叫symmetric呢？因为他俩都是用对应的聚类中心来计算距离，所以是对称的。SDC有一个很大的优点，两两聚类中心之间的距离，是我们在构建向量库的时候就可以知道的，完全可以离线就计算好，在线要计算 $q(x)$ 和 $q(y)$ 的距离的时候，直接查表，提升了在线query的效率。但是，他的误差也比ADC来的大，因为有x和 $q(x)$ ，y和 $q(y)$ 两个量化误差。

ADC

A=asymmetric，不对称的。上文中讲了对称是因为SDC都用了对应的聚类中心。那么ADC，就只有向量库中的y使用了聚类中心，而query向量x没有。那么，计算距离的时候，计算的就是x和 $q(y)$ 的距离了。如图Figure5中的asymmetric case。与SDC不同的是，ADC的精确度更高，因为只有y和 $q(y)$ 这一个量化误差；当然 $\text{dist}(x, q(y))$ 必须要在在线计算(x是用户请求带过来的)，计算速度不如SDC。

那么在实际场景中，一个完整的query向量X是如何跟一个向量库中的样本Y量化之后的结果计算距离的呢？量化之后的样本Y，我们能知道他在每个子空间下的聚类中心，也就是 $q(y)$ ，那么我们就通过ADC或者SDC计算出他在每个子空间下 $q(y)$ 跟x或 $q(x)$ 的距离。我们将每一个子空间下的所有距离的平方相加再开根号，就是最终的X跟Y的距离了(就是使用每个子空间的向量距离进行了一次欧氏距离计算)。如图Figure6所示，以ADC为例。

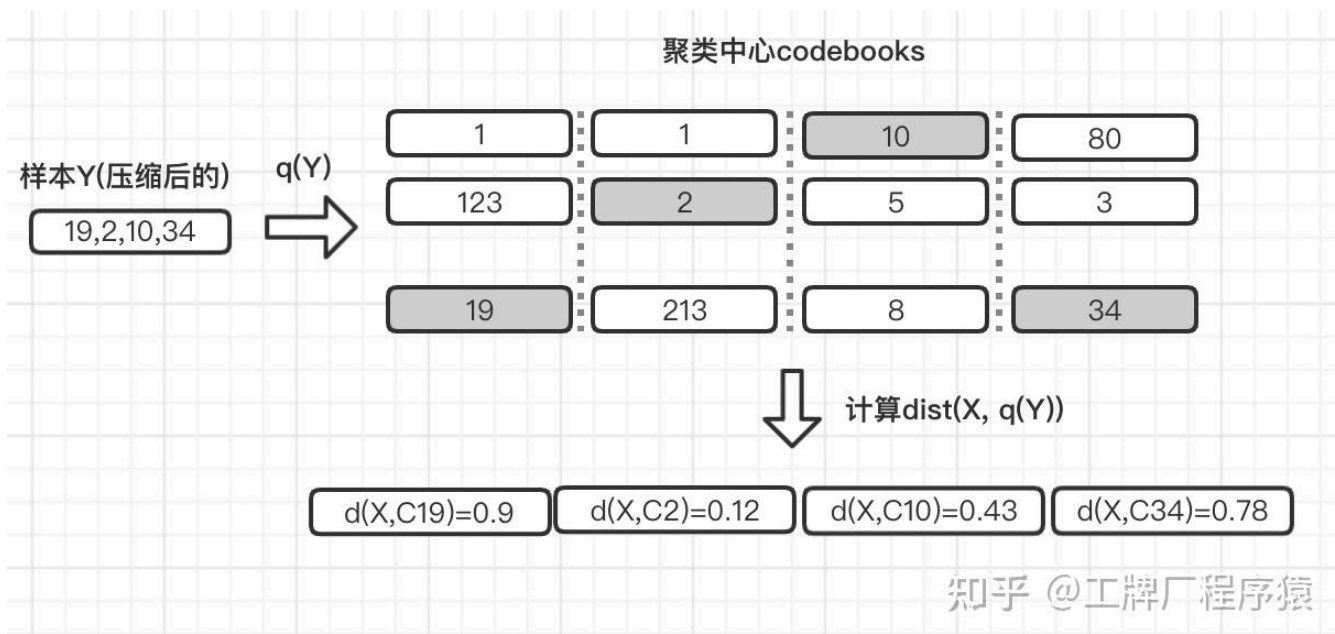


Figure6

$$X \text{ 与样本 } Y \text{ 的距离} = \sqrt{0.9^2 + 0.12^2 + 0.43^2 + 0.78^2} = \sqrt{1.618} = 1.272$$

在 PQ 的场景下，我们要进行检索找出 Top K 临近的样本的话，时间复杂度是 $O(N * M)$ ，因为我们还是要计算出当前样本空间下，查询向量 X 与所有样本 Y 的距离。PQ 只是单纯的压缩算法，检索场景直接用 PQ 还是欠妥。

IVFPQ

接下来就要说说 IVFPQ 了，他做的事情，是在 PQ 的基础之上，解决快速查询的问题。

上文中也提到过，对所有的样本进行距离计算，其实是做了非常多的无用功。那么我们只计算“有潜力”的那几个样本不就好了吗？问题就是如何寻找出“有潜力”的那几个样本呢？最常见的方式就是聚类+倒排。但凡是一个检索系统，倒排必不可少。通过聚类后构建当前类别下所有的样本，我们能在极短的时间内 ($O(K)$)， K 是倒排聚类中心的个数，下文会讲，寻找出 match 当前类别的所有样本。而与当前 query 最为 match 的那个类别，就是“有潜力”的那一批样本。比如这个用户 query 与书籍最为 match，那么它就会命中与书籍相关的那个类别。其实本质很简单，就是分桶。书籍广告往书籍桶里面扔，游戏广告往游戏桶里面扔。IVFPQ 也是如此，所谓 IVF，就是 inverted file system，倒排系统。

既然要分类，那么我们肯定是先聚类。又是 kmeans⁺，我们使用 kmeans 将所有的样本，先进行一次粗粒度的聚类，比如聚类 K 个桶，每个类别有一个自己的聚类中心。此时我们能够知道每个样本 Y 以及它的聚类中心 q(Y)。但是，我们将不会在样本 Y 上直接做 PQ 量化，而是对 Y 和 q(Y) 的残差向量 (向量减法，Y-q(Y)) 做量化。

为什么是用残差？首先我们此时已经对所有的样本做了 kmeans 聚类，那么每个样本跟自己的聚类中心的残差一定是很小的。比如说样本向量 Y 是 [2.4, 2.1]，聚类中心 K 是 [2.3, 2.0]，那么残差就是 $\text{residual} = [2.4 - 2.3, 2.1 - 2.0] = [0.1, 0.1]$ 。我们不难发现，residual 向量比起样本 Y，它的值域要窄得多。值域窄的话，量化误差就会更小，为什么呢？首先依据向量加减法，

$Y - K = \text{residual}$ ，那么 $Y = K + \text{residual}$ 。假设 query 向量为 X，那么 $\text{dist}(X, Y) = \text{dist}(X, K) + \text{dist}(X, \text{residual})$ ，其中，K 是聚类中心，是没有量化过的完整的向量， $\text{dist}(X, K)$ 是没有误差的。对残差向量做量化的话，误差就锁定在了 $\text{dist}(X, \text{residual})$ 。那么我们回到上文中讲述 PQ 量化的时候，也是做了 kmeans 来进行分桶。因为 residual 的值域很小，所以它的方差就很小，那么分桶引发的误差，就会因为值域小而小得多。举个例子，我现在有两组向量，第一组是 [100.2, 200.4], [400.8, 103.2]；第二组是 [0.2, 0.5], [0.9, 0.8]。很显然，第一组的值域比第二组大得多，同样的，第一组两个向量的差距也比第二组大得多。

如图 Figure7，描述了 IVFPQ 的倒排构建过程。

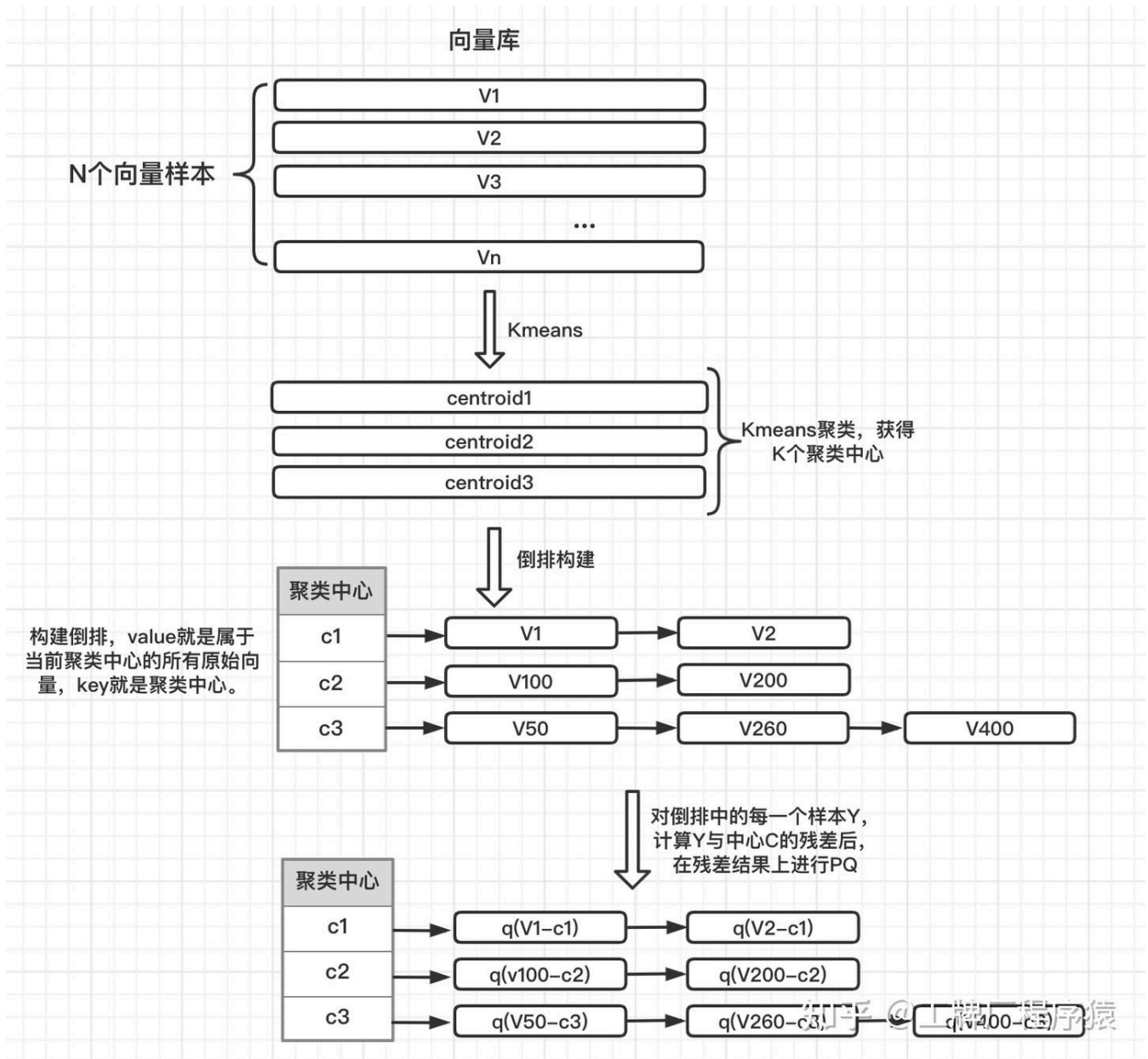


Figure7

那么检索过程, 就非常简单了。query向量X来了, $q(x)$ 计算出属于哪个聚类中心, 比如属于c1, 那么计算x和c1的残差向量之后, 应用上文讲述的SDC或者ADC, 计算出当前类别下, 所有量化后的残差向量跟 $residual = (x - c1)$ 的距离, 就好了。时间复杂度, 仅为 $O(K * D + \frac{N}{K} * M)$, 也就是先找到最合适的聚类中心c, 然后再在这个聚类内部, 寻找 top item。Figure8摘自Paper, 描述了倒排构建和检索流程(基于ADC)。

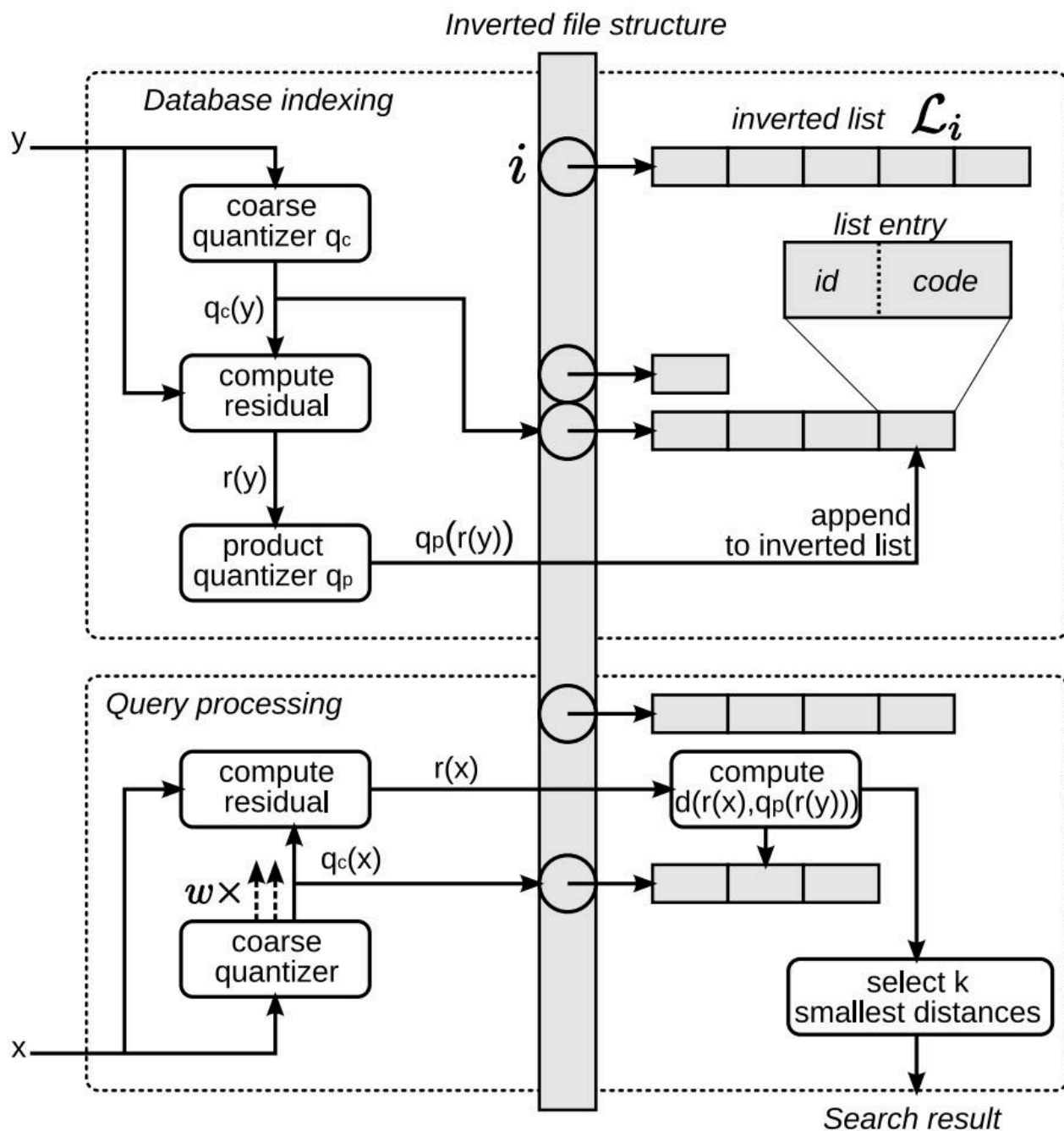


Fig. 5. Overview of the *inverted file with asymmetric distance computation* (IVFADC) indexing system. *Top*: insertion of a vector. *Bottom*: search.

知乎 @工牌厂程序猿

Figure8

现实生产环境中，通常也不会使用纯IVFPQ，因为它毕竟很粗糙。业务方通常会使用IVFPQ加brute-force。比如用IVFPQ粗略地找出Top K条，然后再在这K条中进行暴力计算，找出Top M条作为最终结果。K和M怎么定，通常基于线上AB实验决定。

Reference

lear.inrialpes.fr/pu/

